

Supervised Dimension Reduction

R. Porcedda, F. Chiaromonte (special thanks to J. Di Iorio, L. Insolia and L. Testa)

March 12th 2026

Contents

Introduction	1
Libraries	1
Data	1
PCA vs LDA	3
PCA	3
LDA	3
SIR	5

Introduction

Libraries

We are going to use:

```
library(tidyverse) # data manipulation and visualization
library(plotly)    # plots in 3D
library(ggplot2)   # plots in 2D
library(ggpubr)     # to combine multiple ggplot objects (ggarrange)
library(mvtnorm)    # to generate multivariate normal distribution
library(dr)         # SIR
library(factoextra) # PCA-related functions
```

Data

Let's first define a function to generate Gaussian data. This function takes four arguments:

- n: number of observations;
- center: the mean vector
- sigma: the covariance matrix
- label: the cluster label

```
generateGaussianData <- function(n, center, sigma, label) {
  data = rmvnorm(n, center, sigma)
  data = data.frame(data)
  names(data) = c("x", "y", "z")
  data = data %>% mutate(class=factor(label))
  data
}
```

Now let's simulate a dataset.

```
covmat <- matrix(c(1,0.88,0.88,0.88, 1,0.88,0.88,0.88, 1),
  nrow = 3, byrow=T)
```

```
# cluster 1
```

```
n = 200
```

```
center = c(2, 8, 6)
```

```
sigma = covmat
```

```
group1 = generateGaussianData(n, center, sigma, 1)
```

```
# cluster 2
```

```
n = 200
```

```
center = c(4, 8, 6)
```

```
sigma = covmat
```

```
group2 = generateGaussianData(n, center, sigma, 2)
```

```
# cluster 3
```

```
n = 200
```

```
center = c(6, 8, 6)
```

```
sigma = covmat
```

```
group3 = generateGaussianData(n, center, sigma, 3)
```

```
# all data
```

```
df = bind_rows(group1, group2, group3)
```

```
head(df)
```

```
##           x           y           z class
## 1 1.3660667 7.317555 5.244533      1
## 2 2.8958564 8.884481 6.645029      1
## 3 0.8609425 7.465226 5.116259      1
## 4 2.1916489 8.593764 6.522561      1
## 5 4.8509131 9.708133 7.503172      1
## 6 2.0333742 7.688139 6.752591      1
```

```
summary(df)
```

```
##           x           y           z      class
##  Min.   :-0.8184   Min.    : 5.074   Min.    :3.235   1:200
##  1st Qu.: 2.4187   1st Qu.: 7.347   1st Qu.:5.290   2:200
##  Median : 3.9313   Median : 8.048   Median :5.973   3:200
##  Mean   : 3.9571   Mean    : 8.013   Mean    :5.985
##  3rd Qu.: 5.6324   3rd Qu.: 8.669   3rd Qu.:6.688
##  Max.    : 9.1406   Max.    :10.719   Max.    :8.835
```

And plot our simulated data.

```
fig <- plot_ly(df, x = ~x, y = ~y, z = ~z,
  color = ~class, colors = c('#b3e378', '#81e5f0', '#ed5391'))
```

```
fig <- fig %>% add_markers()
```

```
fig <- fig %>% layout(scene = list(xaxis = list(title = 'x'),
  yaxis = list(title = 'y'),
  zaxis = list(title = 'z')))
```

```
fig
```

PCA vs LDA

PCA

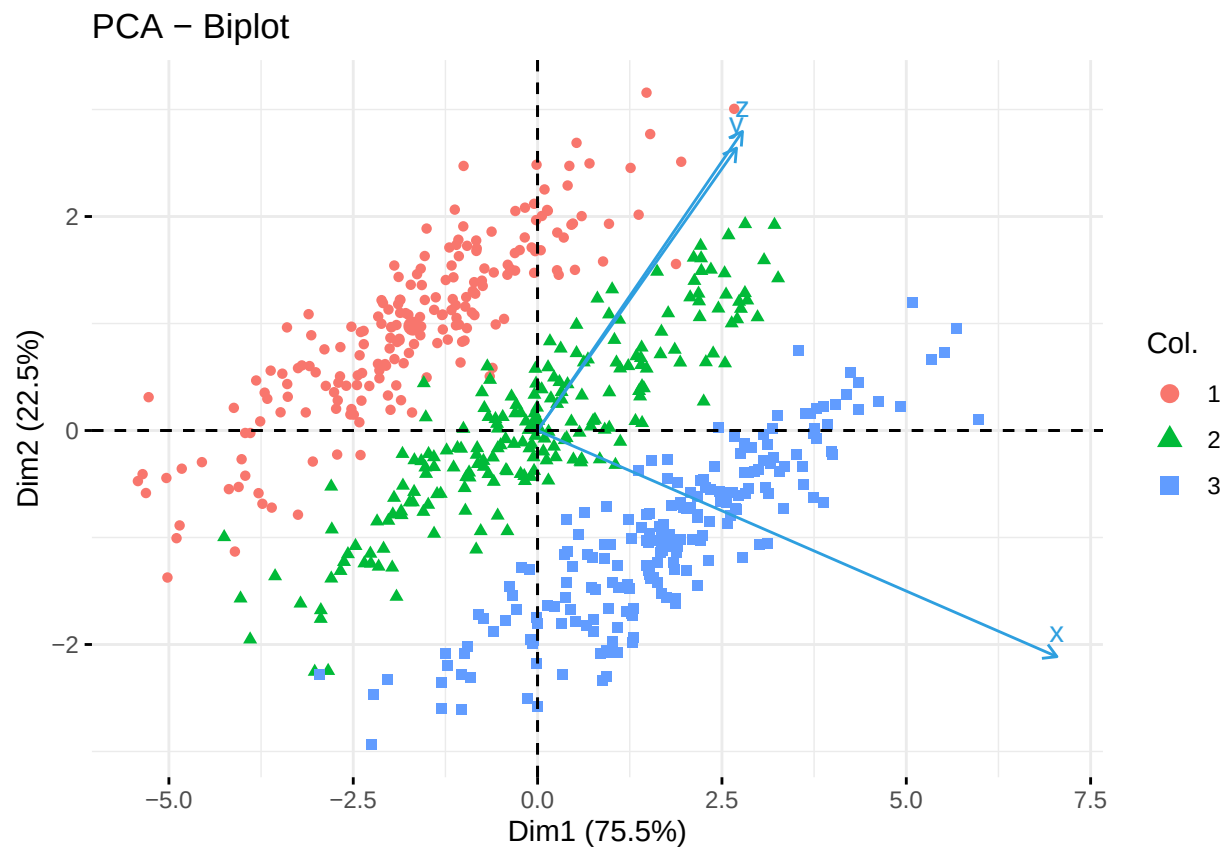
Now let us perform PCA.

```
pc <- prcomp(df[,c(1,2,3)])  
get_eig(pc)
```

```
##      eigenvalue variance.percent cumulative.variance.percent  
## Dim.1  4.4689060       75.524393           75.52439  
## Dim.2  1.3317902       22.507219           98.03161  
## Dim.3  0.1164728        1.968388          100.00000
```

This is the corresponding biplot.

```
fviz_pca_biplot(pc, col.var= "#2E9FDF", col.ind= df$class, label="var")
```



Note that considering the first two principal components it is impossible to notice differences within the three groups (all groups are overlapping).

LDA

Let's perform LDA:

```
lda.df <- lda(factor(class) ~ x + y + z, data = df)  
lda.df
```

```
## Call:  
## lda(factor(class) ~ x + y + z, data = df)
```

```
##
## Prior probabilities of groups:
##      1      2      3
## 0.3333333 0.3333333 0.3333333
##
## Group means:
##      x      y      z
## 1 2.004461 8.029252 6.024801
## 2 3.900658 8.027599 5.987335
## 3 5.966146 7.983131 5.943243
##
## Coefficients of linear discriminants:
##      LD1      LD2
## x  2.307253  0.0119883
## y -1.070498 -2.1710962
## z -1.130563  1.8262917
##
## Proportion of trace:
## LD1 LD2
##   1   0
```

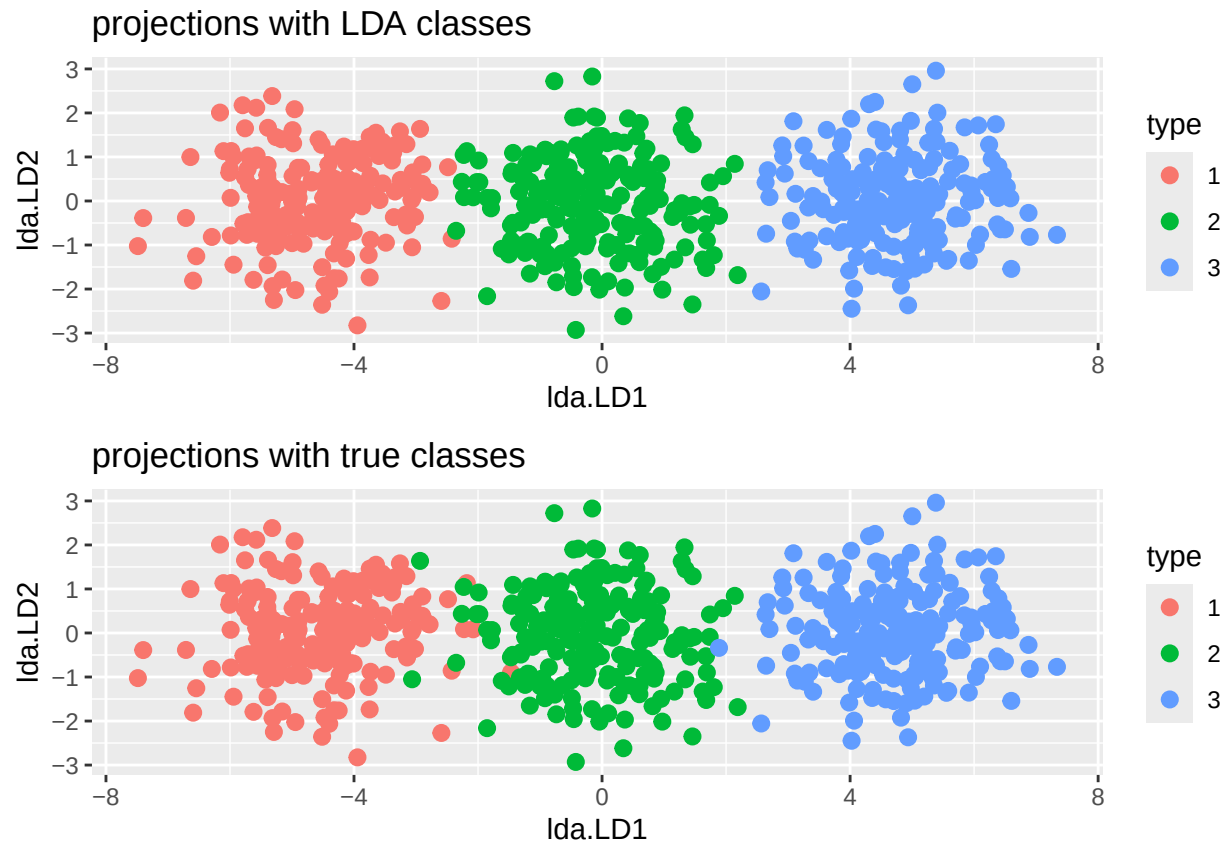
Let us plot the projections on LD1 and LD2

```
# prediction on df to get projections
predmodel.lda = predict(lda.df, data=df)

# projections with LDA classes
estclass <- as.factor(apply(predmodel.lda$posterior, 1, which.max))
newdata2 <- data.frame(type = estclass, lda = predmodel.lda$x)
p1 <- ggplot(newdata2) +
  geom_point(aes(lda.LD1, lda.LD2, colour = type), size = 2.5) +
  ggtitle("projections with LDA classes")

# projections with true classes
newdata <- data.frame(type = df$class, lda = predmodel.lda$x)
p2 <- ggplot(newdata) +
  geom_point(aes(lda.LD1, lda.LD2, colour = type), size = 2.5) +
  ggtitle("projections with true classes")

ggarrange(p1,p2,
  nrow=2)
```



SIR

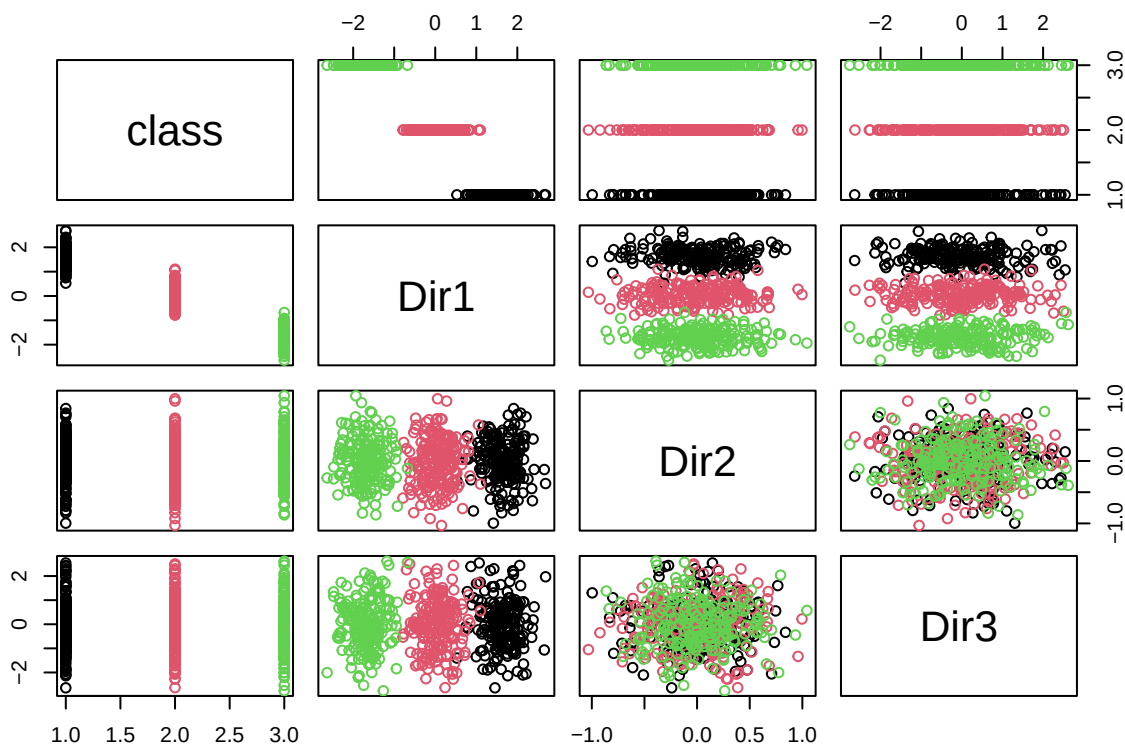
Now we use the SIR (Sliced Inversion Regression) in the dr package

```
# default fitting method is "sir"
help(dr)

dr_res <- dr(class ~ x + y + z, data = df, method='sir')

dr_res

##
## dr(formula = class ~ x + y + z, data = df, method = "sir")
## Estimated Basis Vectors for Central Subspace:
##      Dir1      Dir2      Dir3
## x 0.8289206 0.004225546 0.01962988
## y -0.3845948 -0.765251560 -0.07665615
## z -0.4061742 0.643717481 0.99686434
## Eigenvalues:
## [1] 9.352458e-01 3.837308e-04 1.716338e-16
plot(dr_res, col=df$class)
```

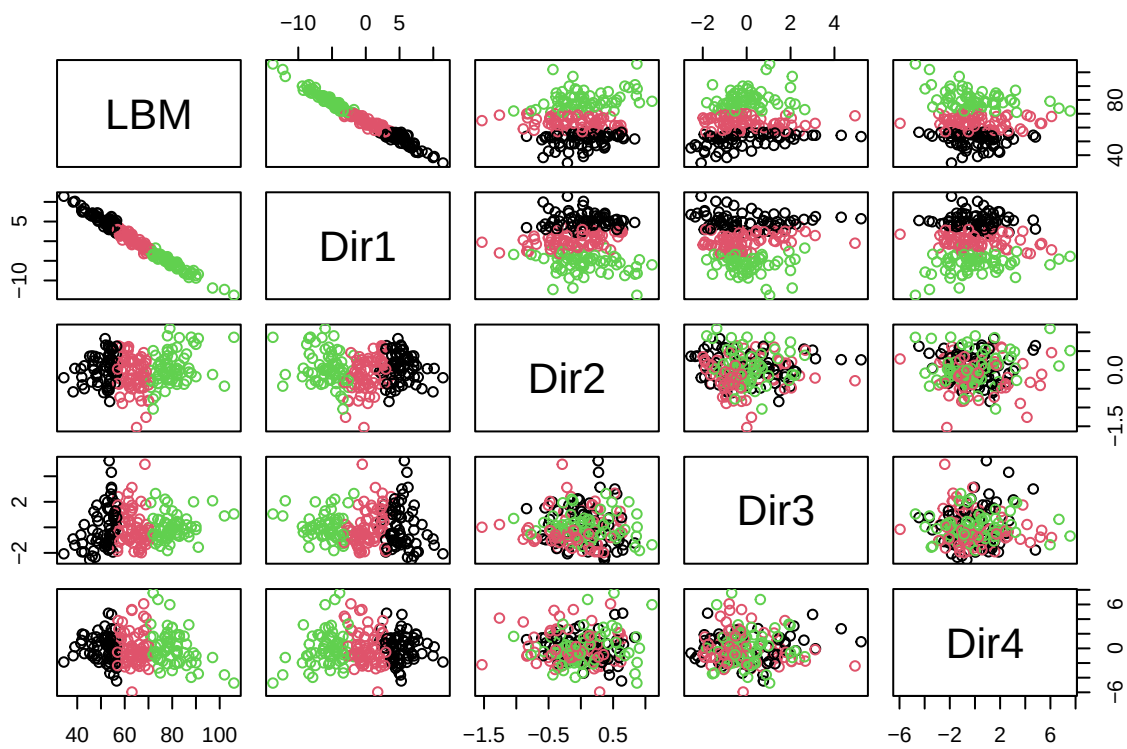


```
names(dr_res)
```

```
## [1] "x"          "y"          "weights"    "method"    "cases"
## [6] "qr"         "group"      "chi2approx" "evectors"  "evalues"
## [11] "numdir"     "raw.evectors" "M"         "slice.info" "call"
## [16] "y.name"     "terms"
```

We perform SIR on real data with continuous outcome

```
data(ais)
?ais
dr_res3 <-dr(LBM~SSF+Wt+Hg+Ht+WCC+RCC+
             Hc+Ferr,data=ais, nslices=3)
plot(dr_res3, col=dr_res3$slice.info$slice.indicator)
```



```
dr_res6 <-dr(LBM~SSF+Wt+Hg+Ht+WCC+RCC+
             Hc+Ferr,data=ais, nslices=6)
plot(dr_res6, col=dr_res6$slice.info$slice.indicator)
```

